

# Simulación de Tormenta mediante Billboards y Sistemas de Partículas utilizando OpenGL

Diego Raúl Rivas Huanca  
Alvarez Astete Jheeremy Manuel

21 de julio de 2026

# Índice general

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1. Introducción</b>                           | <b>4</b>  |
| <b>2. Objetivos</b>                              | <b>6</b>  |
| 2.1. Objetivo General . . . . .                  | 6         |
| 2.2. Objetivos Específicos . . . . .             | 6         |
| <b>3. Fundamento Teórico</b>                     | <b>8</b>  |
| 3.1. Gráficos por Computadora . . . . .          | 8         |
| 3.2. OpenGL . . . . .                            | 8         |
| 3.3. Pipeline Gráfico . . . . .                  | 9         |
| 3.4. Shaders . . . . .                           | 9         |
| 3.4.1. Vertex Shader . . . . .                   | 9         |
| 3.4.2. Fragment Shader . . . . .                 | 10        |
| 3.5. Transformaciones Geométricas . . . . .      | 10        |
| 3.6. Texturas . . . . .                          | 10        |
| 3.7. Alpha Blending . . . . .                    | 11        |
| 3.8. Billboards . . . . .                        | 11        |
| 3.9. Sistema de Partículas . . . . .             | 11        |
| 3.10. Renderizado en Tiempo Real . . . . .       | 12        |
| 3.11. Interfaz Gráfica . . . . .                 | 12        |
| 3.12. Audio Ambiental . . . . .                  | 13        |
| 3.13. Sistema Climático . . . . .                | 13        |
| <b>4. Diseño e Implementación</b>                | <b>14</b> |
| 4.1. Arquitectura General del Proyecto . . . . . | 14        |
| 4.2. Arquitectura por Módulos . . . . .          | 15        |
| 4.3. Clase Application . . . . .                 | 16        |
| 4.4. Ciclo Principal . . . . .                   | 17        |

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| 4.5.      | Renderer . . . . .                                 | 18        |
| 4.6.      | Scene . . . . .                                    | 18        |
| 4.7.      | SceneObject . . . . .                              | 18        |
| 4.8.      | Transform . . . . .                                | 19        |
| 4.9.      | Cámara . . . . .                                   | 19        |
| <b>5.</b> | <b>Implementación</b>                              | <b>20</b> |
| 5.1.      | Implementación del sistema de Billboards . . . . . | 20        |
| 5.2.      | Shaders para Billboards . . . . .                  | 21        |
| 5.3.      | Implementación del sistema de partículas . . . . . | 22        |
| 5.3.1.    | Clase RainParticle . . . . .                       | 22        |
| 5.3.2.    | Clase ParticleEmitter . . . . .                    | 22        |
| 5.3.3.    | Clase RainSystem . . . . .                         | 23        |
| 5.4.      | Implementación del escenario . . . . .             | 23        |
| 5.4.1.    | Terreno (Ground) . . . . .                         | 24        |
| 5.4.2.    | Sistema de árboles . . . . .                       | 24        |
| 5.4.3.    | Distribución aleatoria . . . . .                   | 25        |
| 5.4.4.    | Escalado aleatorio . . . . .                       | 25        |
| 5.4.5.    | Reubicación dinámica . . . . .                     | 25        |
| 5.5.      | Sistema de audio . . . . .                         | 26        |
| 5.5.1.    | Clase AudioManager . . . . .                       | 26        |
| 5.5.2.    | Sincronización con la lluvia . . . . .             | 27        |
| 5.6.      | Sistema meteorológico . . . . .                    | 27        |
| 5.6.1.    | Simulación de relámpagos . . . . .                 | 27        |
| 5.6.2.    | Integración con el renderizador . . . . .          | 27        |
| 5.7.      | Interfaz gráfica . . . . .                         | 28        |
| 5.7.1.    | Panel de control . . . . .                         | 28        |
| 5.7.2.    | Comunicación con el sistema . . . . .              | 29        |
| 5.8.      | Integración del sistema . . . . .                  | 29        |
| 5.8.1.    | Ciclo principal . . . . .                          | 29        |
| 5.8.2.    | Arquitectura final . . . . .                       | 30        |
| <b>6.</b> | <b>Resultados</b>                                  | <b>31</b> |
| 6.1.      | Escena final . . . . .                             | 31        |
| 6.2.      | Sistema de partículas . . . . .                    | 32        |
| 6.3.      | Sistema de árboles . . . . .                       | 32        |

|                                           |           |
|-------------------------------------------|-----------|
| 6.4. Terreno . . . . .                    | 33        |
| 6.5. Interfaz gráfica . . . . .           | 33        |
| 6.6. Sistema de audio . . . . .           | 33        |
| 6.7. Sistema meteorológico . . . . .      | 34        |
| 6.8. Evaluación del rendimiento . . . . . | 34        |
| 6.9. Discusión . . . . .                  | 34        |
| <b>7. Conclusiones</b>                    | <b>36</b> |
| <b>8. Trabajo futuro</b>                  | <b>38</b> |
| <b>9. Repositorio</b>                     | <b>40</b> |

# Capítulo 1

## Introducción

La representación de fenómenos atmosféricos en tiempo real constituye uno de los problemas clásicos dentro del área de gráficos por computadora. Efectos como la lluvia, la nieve, la niebla o el humo pueden involucrar miles de elementos visibles de manera simultánea, lo que representa un desafío importante para el rendimiento de una aplicación gráfica. Una implementación basada en geometría tridimensional tradicional incrementaría considerablemente el número de polígonos procesados por la GPU, reduciendo la velocidad de renderizado y limitando la capacidad de mantener una tasa de cuadros estable.

Una de las técnicas más utilizadas para resolver este problema consiste en el empleo de *billboards*. Un billboard es un plano bidimensional texturizado que modifica automáticamente su orientación para permanecer siempre visible desde la posición de la cámara. Esta técnica permite representar objetos visualmente complejos utilizando únicamente un cuadrilátero, disminuyendo significativamente la carga de procesamiento sin afectar de manera apreciable la percepción visual del usuario.

Además del uso de billboards, los motores gráficos modernos emplean sistemas de partículas para simular fenómenos naturales. Un sistema de partículas administra la creación, actualización y reutilización de una gran cantidad de elementos independientes, permitiendo representar efectos dinámicos como lluvia, fuego, humo, polvo o explosiones. Debido a que cada partícula posee un ciclo de vida propio, el sistema puede reciclar continuamente los recursos gráficos disponibles, logrando un alto rendimiento incluso cuando se representan miles de partículas de manera simultánea.

En este proyecto se desarrolló una aplicación interactiva utilizando OpenGL 3.3 y el lenguaje C++, cuyo objetivo principal consiste en simular una tormenta de lluvia en tiempo real mediante la utilización de billboards y un sistema de partículas. La escena incorpora además un terreno texturizado, árboles implementados mediante billboards,

un sistema básico de clima con relámpagos, sonido ambiental de lluvia y una interfaz gráfica desarrollada con Dear ImGui que permite modificar diversos parámetros de ejecución en tiempo real.

Durante el desarrollo se adoptó una arquitectura modular basada en componentes independientes, separando claramente las responsabilidades de renderizado, administración de escenas, recursos gráficos, partículas, cámara, interfaz gráfica, sistema de audio y efectos climáticos. Esta organización facilita la reutilización del código, mejora la mantenibilidad del proyecto y permite incorporar nuevas funcionalidades sin modificar significativamente los módulos existentes.

Finalmente, el proyecto demuestra que la combinación de billboards, sistemas de partículas y una adecuada administración de recursos constituye una solución eficiente para la representación de fenómenos atmosféricos en aplicaciones gráficas en tiempo real, manteniendo un equilibrio entre calidad visual y rendimiento computacional.

# Capítulo 2

## Objetivos

### 2.1. Objetivo General

Desarrollar una aplicación gráfica interactiva en tiempo real utilizando OpenGL 3.3 y el lenguaje C++, capaz de simular una escena de lluvia mediante el uso de billboards y sistemas de partículas, incorporando además efectos ambientales, una interfaz gráfica de control y una arquitectura modular que facilite la escalabilidad y el mantenimiento del proyecto.

### 2.2. Objetivos Específicos

- Diseñar e implementar un sistema de renderizado basado en OpenGL para la representación de objetos bidimensionales y tridimensionales dentro de una escena virtual.
- Implementar un sistema de partículas que permita generar, actualizar y reciclar miles de partículas de lluvia de forma eficiente.
- Utilizar la técnica de billboards para representar las gotas de lluvia y los árboles, logrando una reducción significativa en el número de polígonos procesados sin afectar la percepción visual del escenario.
- Desarrollar un terreno texturizado que sirva como superficie base para la simulación del entorno.
- Implementar una cámara con movimiento libre sobre el plano horizontal que permita recorrer la escena de manera intuitiva.

- Incorporar una interfaz gráfica mediante Dear ImGui que permita modificar en tiempo real parámetros como la cantidad de partículas, velocidad de la lluvia y dirección del viento.
- Integrar un sistema de audio ambiental utilizando la biblioteca miniaudio para reproducir sonidos de lluvia y mejorar la inmersión del usuario.
- Implementar un sistema climático simple que genere relámpagos mediante variaciones temporales de iluminación dentro de la escena.
- Diseñar una arquitectura modular separando los diferentes componentes del sistema en módulos independientes para facilitar su mantenimiento y futura expansión.
- Evaluar el funcionamiento del sistema verificando la correcta integración de todos los módulos y el desempeño de la aplicación bajo diferentes configuraciones de partículas.



# Capítulo 3

## Fundamento Teórico

### 3.1. Gráficos por Computadora

Los gráficos por computadora constituyen una rama de la informática dedicada a la generación y manipulación de imágenes mediante algoritmos computacionales. En aplicaciones interactivas, como videojuegos o simuladores, el objetivo principal consiste en representar escenas tridimensionales manteniendo una velocidad de renderizado suficientemente alta para proporcionar una experiencia fluida al usuario.

Actualmente, la mayor parte del procesamiento gráfico es realizado por la Unidad de Procesamiento Gráfico (GPU), dispositivo especializado en ejecutar operaciones matemáticas de manera paralela sobre miles de vértices y fragmentos simultáneamente. Para aprovechar esta capacidad, las aplicaciones utilizan interfaces de programación como OpenGL, DirectX o Vulkan.

En este proyecto se empleó OpenGL debido a su portabilidad, amplia documentación y compatibilidad con múltiples plataformas.

### 3.2. OpenGL

OpenGL (Open Graphics Library) es una API estándar para el desarrollo de aplicaciones gráficas en dos y tres dimensiones. Su función principal consiste en proporcionar un conjunto de instrucciones que permiten enviar información geométrica hacia la GPU para ser procesada mediante el pipeline gráfico.

OpenGL no genera escenas automáticamente; únicamente proporciona las herramientas necesarias para que el programador defina vértices, texturas, materiales, transformaciones, iluminación y sombreadores.

En este proyecto se utilizó OpenGL 3.3 Core Profile, versión que elimina funciones antiguas del pipeline fijo y obliga a implementar los procesos gráficos mediante shaders programables.

### 3.3. Pipeline Gráfico

El pipeline gráfico representa la secuencia de etapas mediante las cuales un conjunto de vértices termina convirtiéndose en una imagen visible sobre la pantalla.

Las principales etapas del pipeline utilizado durante el desarrollo fueron:

- Definición de la geometría mediante vértices.
- Transformación de coordenadas utilizando matrices de modelo, vista y proyección.
- Procesamiento en el Vertex Shader.
- Rasterización.
- Procesamiento de fragmentos mediante el Fragment Shader.
- Escritura del resultado en el framebuffer.

Cada cuadro generado por la aplicación sigue exactamente esta secuencia.

### 3.4. Shaders

Los shaders son pequeños programas ejecutados directamente por la GPU. Su objetivo consiste en procesar la información gráfica de forma altamente paralela.

En este proyecto se emplearon dos tipos de shaders.

#### 3.4.1. Vertex Shader

El Vertex Shader procesa individualmente cada vértice de un objeto. En él se aplican las transformaciones espaciales utilizando las matrices de modelo, vista y proyección.

Además, para los billboards se calcula la orientación del plano utilizando los vectores Right y Up obtenidos desde la cámara.

### 3.4.2. Fragment Shader

Después de la rasterización, el Fragment Shader determina el color final de cada píxel.

En este proyecto los fragment shaders realizan principalmente dos tareas:

- Aplicar las texturas.
- Descartar píxeles completamente transparentes mediante la instrucción **discard**.

Gracias a ello es posible utilizar imágenes PNG con transparencia sin que aparezca el cuadrado completo del billboard.

## 3.5. Transformaciones Geométricas

Cada objeto de la escena posee una transformación independiente formada por tres componentes fundamentales:

- Posición.
- Rotación.
- Escala.

Estas transformaciones son almacenadas dentro de una matriz de modelo (*Model Matrix*), la cual convierte las coordenadas locales del objeto hacia coordenadas del mundo.

Posteriormente, la matriz de vista transforma dichas coordenadas al sistema de referencia de la cámara, mientras que la matriz de proyección convierte la escena tridimensional en coordenadas bidimensionales aptas para ser mostradas sobre la pantalla.

## 3.6. Texturas

Una textura es una imagen bidimensional utilizada para aportar detalle visual a una geometría sencilla.

Durante el proyecto se utilizaron texturas PNG con canal alfa para representar:

- Gotas de lluvia.
- Árboles.

- Terreno.

El canal alfa permite representar únicamente las regiones visibles de la imagen, descartando automáticamente el fondo transparente.

### 3.7. Alpha Blending

El Alpha Blending permite combinar el color de un fragmento con el color previamente almacenado en el framebuffer.

La configuración utilizada fue:

```
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
```

Esta configuración resulta adecuada para objetos parcialmente transparentes como lluvia, humo o vegetación.

### 3.8. Billboards

Los billboards constituyen una técnica ampliamente utilizada para representar objetos complejos mediante un único cuadrilátero texturizado.

Su funcionamiento consiste en orientar continuamente dicho plano hacia la cámara para que el observador siempre perciba la textura de frente.

Esta técnica permite representar vegetación, partículas, explosiones, humo o efectos atmosféricos reduciendo considerablemente el número de polígonos procesados.

En este proyecto los billboards fueron utilizados para representar tanto las gotas de lluvia como los árboles del escenario.

### 3.9. Sistema de Partículas

Un sistema de partículas administra un conjunto de elementos independientes que comparten un comportamiento similar.

Cada partícula posee información propia, entre la que se encuentra:

- Posición.
- Velocidad.

- Dirección.
- Estado.
- Tiempo de vida.

Cuando una partícula alcanza el final de su ciclo de vida, el sistema la reutiliza asignándole una nueva posición inicial. Este mecanismo evita crear y destruir objetos continuamente, reduciendo significativamente el costo computacional.

### 3.10. Renderizado en Tiempo Real

Una aplicación interactiva ejecuta continuamente un ciclo conocido como *Game Loop*.

Durante cada iteración se realizan las siguientes operaciones:

1. Procesamiento de entradas del usuario.
2. Actualización de la lógica.
3. Actualización de partículas.
4. Actualización de la cámara.
5. Renderizado completo de la escena.
6. Presentación del frame generado.

Este ciclo se repite decenas de veces por segundo hasta finalizar la aplicación.

### 3.11. Interfaz Gráfica

Para permitir la modificación de parámetros durante la ejecución se utilizó Dear ImGui.

Esta biblioteca proporciona una interfaz inmediata mediante la cual fue posible controlar:

- Cantidad de partículas.
- Velocidad de la lluvia.
- Intensidad del viento.

- Información de rendimiento (FPS).
- Posición de la cámara.

Gracias a ello el comportamiento del sistema puede modificarse sin necesidad de recompilar la aplicación.

### **3.12. Audio Ambiental**

La ambientación sonora se implementó utilizando la biblioteca Miniaudio.

El sistema reproduce un sonido continuo de lluvia cuya intensidad aumenta conforme incrementa la cantidad de partículas presentes en la escena.

Adicionalmente, el diseño del proyecto permite incorporar fácilmente nuevos sonidos ambientales, como viento, truenos o efectos climáticos adicionales.

### **3.13. Sistema Climático**

Como complemento visual se implementó un sistema climático sencillo encargado de generar relámpagos.

El efecto consiste en incrementar temporalmente la intensidad de iluminación del fondo mediante una interpolación controlada por tiempo, simulando el destello característico producido durante una tormenta.

Aunque se trata de una implementación simple, contribuye significativamente a mejorar la inmersión del escenario.

# Capítulo 4

## Diseño e Implementación

### 4.1. Arquitectura General del Proyecto

Durante el desarrollo se optó por una arquitectura modular, en la cual cada componente del sistema posee una responsabilidad específica. Esta organización permite mantener un bajo acoplamiento entre módulos, facilitando tanto el mantenimiento del código como la incorporación de nuevas funcionalidades.

La estructura final del proyecto quedó organizada como se muestra en la Figura 4.1.

```
cg-billboard-rain

assets
  audio
  shaders
  textures

docs

external

src
  audio
  core
  graphics
  gui
  math
  particles
  resources
  scene
  weather
  utils

CMakeLists.txt
README.md
```

Figura 4.1: Estructura general del proyecto.

Cada directorio contiene un conjunto de clases relacionadas con una funcionalidad específica del sistema.

## 4.2. Arquitectura por Módulos

La Figura 4.2 presenta la arquitectura general del sistema desarrollado. La clase `Application` coordina todos los subsistemas, mientras que cada módulo posee una responsabilidad específica dentro de la simulación.



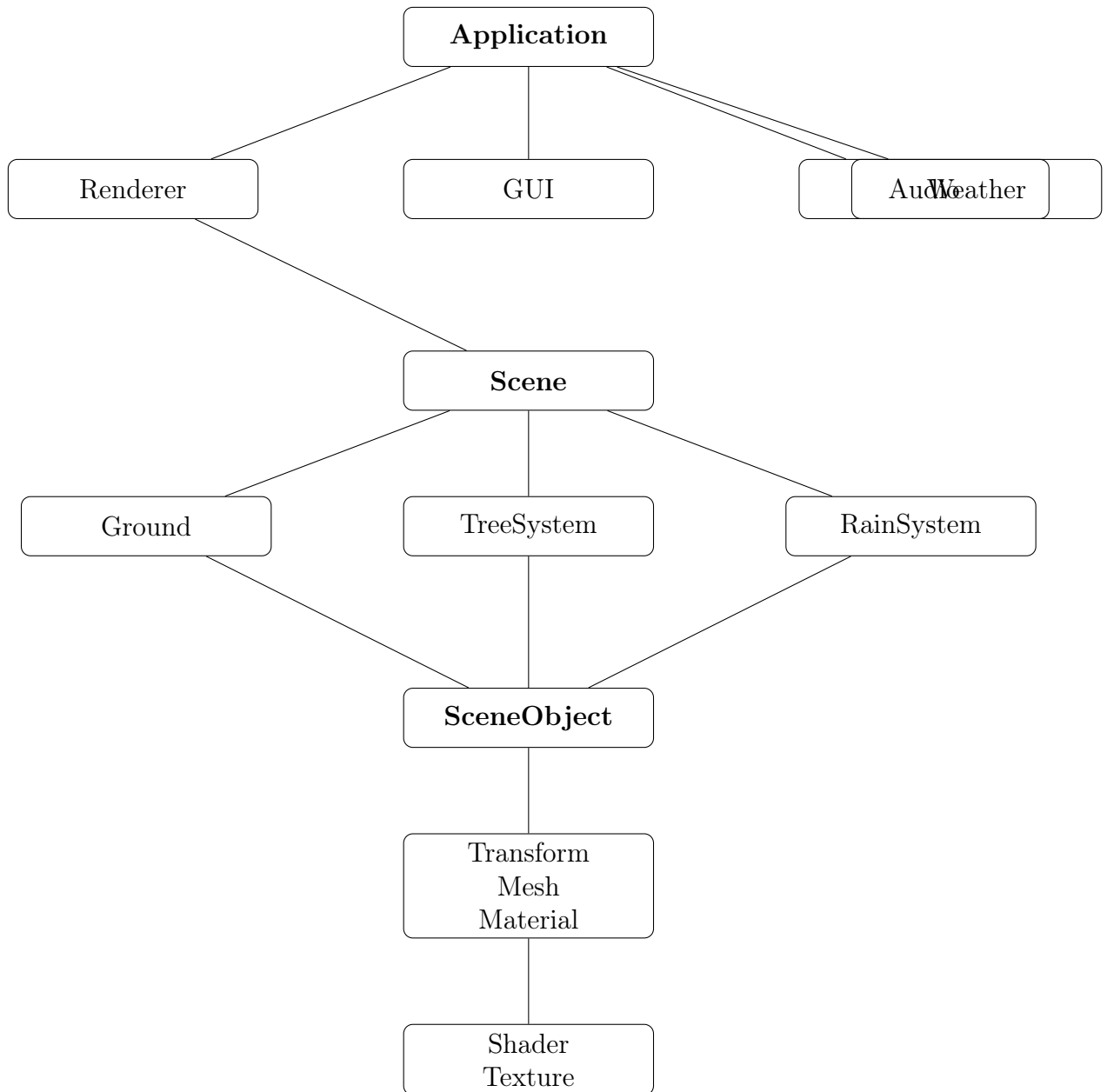


Figura 4.2: Arquitectura general del sistema desarrollado.

La clase **Application** constituye el punto central del sistema y coordina el funcionamiento de todos los módulos.

### 4.3. Clase **Application**

La clase **Application** representa el núcleo del programa. Su responsabilidad consiste en inicializar todos los subsistemas, ejecutar el ciclo principal de la aplicación y liberar correctamente los recursos al finalizar la ejecución.

Durante la fase de inicialización se realizan las siguientes tareas:

- Creación de la ventana mediante GLFW.
- Inicialización del renderer.
- Inicialización del sistema de audio.
- Inicialización de Dear ImGui.
- Construcción del escenario.
- Inicialización del sistema de lluvia.
- Inicio del sonido ambiental.

Posteriormente comienza el ciclo principal del programa, mostrado en la Figura ??.

## 4.4. Ciclo Principal

La aplicación ejecuta continuamente un ciclo hasta que el usuario decide cerrar la ventana.

Cada iteración realiza las siguientes operaciones:

1. Lectura del teclado.
2. Actualización de la cámara.
3. Actualización del terreno.
4. Actualización del sistema de árboles.
5. Actualización del sistema de lluvia.
6. Actualización del sistema de audio.
7. Actualización de la interfaz gráfica.
8. Renderizado de toda la escena.

Este proceso garantiza que todos los elementos permanezcan sincronizados durante la ejecución.

## 4.5. Renderer

El módulo Renderer encapsula todas las operaciones relacionadas con OpenGL.

Entre sus responsabilidades se encuentran:

- Configuración del viewport.
- Configuración del color de limpieza.
- Activación del Depth Test.
- Activación del Alpha Blending.
- Envío de matrices de vista y proyección.
- Envío de los vectores de la cámara.
- Control del efecto de relámpago.

De esta manera, el resto del proyecto nunca realiza llamadas directas a OpenGL.

## 4.6. Scene

La clase Scene actúa como un contenedor de todos los objetos presentes en el escenario.

Cada elemento derivado de SceneObject es almacenado dentro de un vector dinámico.

Durante el renderizado únicamente es necesario recorrer dicho vector para dibujar todos los objetos de manera secuencial.

Esta aproximación simplifica considerablemente la administración de recursos gráficos.

## 4.7. SceneObject

Todos los elementos visibles heredan de la clase SceneObject.

Cada objeto posee tres componentes fundamentales:

- Transform.
- Mesh.

- Material.

Gracias a esta estructura fue posible reutilizar la misma arquitectura para representar lluvia, árboles y terreno sin modificar el renderer.

## 4.8. Transform

Cada objeto almacena su transformación mediante la clase Transform.

Esta clase mantiene la información correspondiente a:

- Posición.
- Rotación.
- Escala.

A partir de estos valores se construye la matriz de modelo enviada posteriormente al Vertex Shader.

La utilización de esta clase permitió modificar cualquier objeto del escenario sin alterar directamente sus vértices.

## 4.9. Cámara

La cámara implementa un sistema de navegación libre sobre el plano XZ.

El usuario puede desplazarse utilizando las teclas W, A, S y D.

Para evitar movimientos verticales no deseados, el vector Forward utilizado para el desplazamiento se proyecta sobre el plano horizontal antes de actualizar la posición de la cámara.

Esta modificación evita que el terreno parezca inclinarse al avanzar y mantiene una navegación más natural dentro del escenario.

# Capítulo 5

## Implementación

En este capítulo se describe el desarrollo de cada uno de los módulos implementados para la simulación de lluvia mediante billboards utilizando OpenGL. La implementación se realizó siguiendo una arquitectura modular, donde cada componente posee una responsabilidad claramente definida, facilitando el mantenimiento y la incorporación de nuevas funcionalidades.

### 5.1. Implementación del sistema de Billboards

El elemento fundamental del proyecto corresponde a los billboards, utilizados para representar tanto las gotas de lluvia como los árboles del escenario.

Un billboard consiste en un plano bidimensional (quad) cuya orientación se actualiza automáticamente para mantenerse siempre de frente a la cámara. Gracias a esta técnica es posible representar objetos aparentemente tridimensionales utilizando únicamente cuatro vértices, reduciendo considerablemente la carga de procesamiento gráfico.

Para ello se implementó la clase `QuadMesh`, encargada de construir la geometría básica utilizada por todos los billboards del proyecto.

Los vértices definidos corresponden a un cuadrado centrado en el origen del sistema de coordenadas local.

- 4 vértices.
- 6 índices (2 triángulos).
- Coordenadas de textura normalizadas.

Cada objeto billboard comparte esta misma geometría, diferenciándose únicamente por su textura, escala y posición.

Esta estrategia permite reutilizar recursos gráficos y disminuir el consumo de memoria de la aplicación.

## 5.2. Shaders para Billboards

El comportamiento de los billboards se implementó mediante un Vertex Shader especializado.

A diferencia de un objeto tridimensional convencional, cuya orientación depende completamente de la matriz de modelo, un billboard calcula su posición utilizando directamente los vectores de orientación de la cámara.

Durante cada cuadro se envían al shader los vectores:

- Right
- Up

A partir de estos vectores se reconstruye la orientación del quad dentro del espacio mundial.

De esta forma, sin importar la dirección en la que observe el usuario, el billboard permanece siempre orientado hacia la cámara.

El cálculo realizado en el shader puede expresarse mediante la siguiente ecuación:

$$P_{world} = C + Right \cdot x + Up \cdot y$$

donde:

- $C$  representa el centro del billboard.
- $Right$  corresponde al eje horizontal de la cámara.
- $Up$  representa el eje vertical de la cámara.
- $(x, y)$  son las coordenadas locales de cada vértice.

Posteriormente, el Fragment Shader aplica la textura correspondiente y descarta automáticamente los píxeles transparentes utilizando el canal alfa.

Este procedimiento permite representar árboles y gotas de lluvia con una apariencia tridimensional utilizando únicamente un plano bidimensional.

## 5.3. Implementación del sistema de partículas

El efecto de lluvia fue implementado mediante un sistema de partículas diseñado específicamente para representar miles de gotas simultáneamente.

La arquitectura desarrollada se divide en tres componentes principales:

- `RainParticle`
- `ParticleEmitter`
- `RainSystem`

Cada uno de ellos posee una responsabilidad específica dentro del ciclo de simulación.

### 5.3.1. Clase `RainParticle`

Cada gota de lluvia se representa mediante una instancia de la clase `RainParticle`. Cada partícula almacena información relacionada con:

- Posición.
- Velocidad.
- Estado de vida.
- Material gráfico.
- Transformación.

Durante cada actualización la posición de la partícula se modifica considerando la velocidad de caída y la influencia del viento.

Cuando la partícula alcanza el suelo, ésta se reinicializa automáticamente sobre el área de emisión, permitiendo reutilizar continuamente los mismos objetos sin necesidad de crear nuevas instancias durante la ejecución.

### 5.3.2. Clase `ParticleEmitter`

La clase `ParticleEmitter` administra todas las partículas existentes.

Su responsabilidad consiste en:

- Crear las partículas.

- Almacenarlas.
- Actualizar cada una de ellas.
- Reiniciar aquellas que alcanzan el suelo.

Esta aproximación evita operaciones frecuentes de asignación dinámica de memoria, mejorando el rendimiento general del sistema.

### 5.3.3. Clase RainSystem

El módulo RainSystem coordina completamente la simulación de lluvia.

Entre sus responsabilidades principales se encuentran:

- Inicializar el emisor.
- Reconstruir el sistema cuando cambia el número de partículas.
- Actualizar la intensidad del viento.
- Actualizar la velocidad de caída.
- Mantener la lluvia centrada alrededor de la cámara.

La lluvia no permanece fija dentro del escenario. En su lugar, el área de emisión acompaña continuamente al usuario, creando la sensación de una precipitación infinita sin necesidad de simular un escenario de grandes dimensiones.

Esta técnica reduce considerablemente el consumo de memoria y mantiene un número constante de partículas activas independientemente de la distancia recorrida por el usuario

## 5.4. Implementación del escenario

Con el objetivo de proporcionar un entorno visual que permitiera apreciar correctamente el efecto de lluvia, se implementó un escenario compuesto por un terreno y un conjunto de árboles distribuidos alrededor del usuario.

El escenario fue desarrollado bajo una arquitectura modular, donde cada elemento administra sus propios recursos gráficos, permitiendo reutilizar modelos, materiales y texturas sin generar duplicados innecesarios en memoria.

La escena completa es administrada mediante la clase **Scene**, la cual mantiene una colección de objetos renderizables y delega su actualización y dibujado al renderizador principal.



### 5.4.1. Terreno (Ground)

El terreno representa la superficie sobre la cual se desarrolla la simulación.

Para su implementación se construyó un plano compuesto únicamente por cuatro vértices y dos triángulos, utilizando una textura repetida múltiples veces para evitar pérdida de calidad al aumentar su tamaño.

Durante el desarrollo se decidió incrementar las dimensiones del plano respecto a la versión inicial para impedir que el usuario observara el borde del escenario al desplazarse.

La textura del terreno utiliza coordenadas UV superiores al rango  $[0, 1]$ , permitiendo que ésta se repita automáticamente sobre toda la superficie mediante la configuración de repetición (*texture wrapping*) de OpenGL.

Adicionalmente, el terreno no permanece completamente fijo dentro del mundo.

En cada actualización su posición se sincroniza con las coordenadas XZ de la cámara, mientras la altura permanece constante.

Esta técnica produce la sensación de un terreno prácticamente infinito utilizando únicamente un único plano renderizado.

### 5.4.2. Sistema de árboles

Los árboles fueron implementados utilizando la misma técnica de billboards empleada para las partículas, aunque con un comportamiento diferente.

Cada árbol corresponde a un plano bidimensional con una textura PNG que incluye canal alfa para eliminar automáticamente el fondo mediante el Fragment Shader.

Todos los árboles comparten:

- La misma geometría.
- El mismo shader.
- El mismo material.
- La misma textura.

Únicamente cambian su posición y escala dentro del escenario.

Esta estrategia reduce significativamente el consumo de memoria gráfica.

### 5.4.3. Distribución aleatoria

Los árboles son generados utilizando posiciones aleatorias dentro de un radio determinado alrededor del origen.

Sin embargo, una distribución completamente aleatoria puede producir superposición entre árboles, disminuyendo el realismo de la escena.

Para evitar este problema se implementó una distancia mínima entre árboles.

Cada nueva posición generada se compara contra todos los árboles existentes.

Si la distancia es menor al umbral establecido, la posición es descartada y se genera una nueva coordenada aleatoria.

Este procedimiento continúa hasta encontrar una posición válida.

Gracias a este algoritmo se obtiene una distribución visualmente uniforme sobre todo el escenario.

### 5.4.4. Escalado aleatorio

Con el propósito de evitar que todos los árboles posean exactamente el mismo tamaño, cada instancia recibe una escala generada aleatoriamente dentro de un intervalo previamente definido.

Esta variación produce una escena más natural y disminuye la percepción de repetición de la textura.

Cada árbol conserva su propia escala incluso cuando es reposicionado posteriormente durante la simulación.

### 5.4.5. Reubicación dinámica

Debido a que el escenario representa un bosque virtualmente infinito, no resulta eficiente generar miles de árboles.

En su lugar se implementó un sistema de reciclaje.

Cada árbol verifica continuamente su distancia respecto a la cámara.

Cuando un árbol supera el radio máximo definido para la simulación, éste es trasladado automáticamente hacia una nueva posición ubicada delante del usuario.

La nueva ubicación incorpora un desplazamiento aleatorio horizontal para evitar patrones repetitivos.

Este procedimiento mantiene aproximadamente constante el número de árboles presentes en memoria mientras genera la sensación de un bosque continuo.

## 5.5. Sistema de audio

Para complementar la experiencia visual se incorporó un sistema de audio utilizando la biblioteca Miniaudio.

Esta biblioteca fue seleccionada debido a que consiste en un único archivo de cabecera, es multiplataforma y no requiere dependencias adicionales para reproducir archivos WAV.

El proyecto organiza todos los recursos de sonido dentro del directorio:

`assets/audio`

Actualmente se utilizan efectos correspondientes a:

- Lluvia continua.
- Truenos.

### 5.5.1. Clase AudioManager

Toda la reproducción sonora se encuentra encapsulada dentro de la clase **AudioManager**.

Esta clase administra la inicialización del motor de audio, la carga de archivos, el control del volumen y la reproducción de efectos.

Entre las funciones implementadas destacan:

- Inicialización del motor.
- Reproducción continua de lluvia.
- Control del volumen.
- Control de la velocidad de reproducción.
- Reproducción de efectos individuales.
- Liberación de recursos.

De esta forma el resto de la aplicación permanece completamente desacoplado de la biblioteca Miniaudio.

### 5.5.2. Sincronización con la lluvia

El sonido de la lluvia responde dinámicamente a la intensidad configurada por el usuario.

Cuando aumenta el número de partículas desde la interfaz gráfica, también incrementa el volumen del sonido y la velocidad de reproducción del efecto ambiental.

Este comportamiento genera una percepción auditiva coherente con la cantidad de lluvia observada en pantalla.

La intensidad se calcula normalizando el número actual de partículas respecto al máximo permitido por la interfaz gráfica.

## 5.6. Sistema meteorológico

Con el propósito de incrementar el realismo de la simulación, además del sistema de partículas se implementó un módulo encargado de representar fenómenos meteorológicos complementarios.

Actualmente dicho módulo incorpora efectos de relámpagos que modifican temporalmente la iluminación general de la escena.

El sistema fue diseñado de forma independiente del renderizador y del sistema de partículas, permitiendo extender fácilmente la simulación mediante nuevos fenómenos atmosféricos como niebla, viento variable o ciclos de día y noche.

### 5.6.1. Simulación de relámpagos

Los relámpagos fueron implementados mediante un incremento temporal de la iluminación del fondo de la escena.

Cuando ocurre un evento de rayo, el sistema incrementa una variable denominada `lightningIntensity`, cuyo valor representa la intensidad instantánea del destello.

Durante cada actualización esta intensidad disminuye progresivamente hasta volver a cero, produciendo una transición suave en lugar de un cambio brusco.

Este comportamiento simula el destello característico de un relámpago observado durante una tormenta.

### 5.6.2. Integración con el renderizador

La intensidad calculada por el sistema meteorológico es enviada al renderizador mediante una interfaz pública.

Antes de limpiar el framebuffer, el renderizador modifica dinámicamente el color utilizado por `glClearColor()`.

Cuando no existe ningún relámpago activo, el fondo conserva un color oscuro que representa una tormenta nocturna.

Durante un destello, la intensidad se suma temporalmente a los componentes RGB del color de fondo, generando una iluminación global de toda la escena.

Debido a que el efecto se aplica directamente durante el proceso de limpieza del framebuffer, no resulta necesario modificar los shaders de los distintos objetos renderizados.

## 5.7. Interfaz gráfica

Para facilitar la interacción con la simulación se incorporó una interfaz gráfica desarrollada utilizando la biblioteca Dear ImGui.

Esta biblioteca permite construir interfaces en tiempo real orientadas principalmente al desarrollo y depuración de aplicaciones gráficas.

Toda la lógica correspondiente a la interfaz fue encapsulada dentro de la clase `GuiManager`, evitando mezclar código de renderizado con la lógica de presentación.

### 5.7.1. Panel de control

La interfaz desarrollada permite modificar diversos parámetros de la simulación durante su ejecución.

Entre las opciones disponibles se encuentran:

- Número de partículas de lluvia.
- Intensidad del viento.
- Velocidad de caída de las partículas.
- Información del rendimiento (FPS).
- Tiempo por cuadro (Frame Time).
- Posición actual de la cámara.

La modificación de cualquiera de estos parámetros produce una actualización inmediata del comportamiento del sistema sin necesidad de reiniciar la aplicación.

### 5.7.2. Comunicación con el sistema

La interfaz no modifica directamente los componentes internos de la simulación.

En su lugar, actúa como un intermediario que proporciona los parámetros configurados por el usuario.

Durante cada iteración del ciclo principal, la aplicación consulta dichos valores y actualiza los distintos módulos correspondientes.

Por ejemplo, un cambio en el número de partículas provoca la reconstrucción del sistema de lluvia y la actualización simultánea de la intensidad del sonido ambiental.

Este diseño mantiene un bajo acoplamiento entre la interfaz gráfica y la lógica principal del programa.

## 5.8. Integración del sistema

Todos los módulos implementados fueron integrados dentro de la clase `Application`, la cual actúa como punto central de coordinación.

Durante la inicialización se crean la ventana, el renderizador, el sistema de audio, la interfaz gráfica, el escenario y el sistema de lluvia.

Posteriormente comienza el ciclo principal de la aplicación, el cual se ejecuta continuamente hasta que el usuario decide cerrar la ventana.

### 5.8.1. Ciclo principal

En cada iteración del ciclo principal se realizan las siguientes etapas:

1. Procesamiento de eventos de entrada mediante GLFW.
2. Actualización del temporizador.
3. Actualización de la cámara.
4. Actualización del terreno.
5. Actualización del sistema de árboles.
6. Actualización del sistema de lluvia.
7. Actualización del sistema de audio.
8. Actualización del sistema meteorológico.

9. Renderizado de toda la escena.
10. Renderizado de la interfaz gráfica.
11. Intercambio de buffers.

Esta estructura mantiene claramente separadas las responsabilidades de actualización y renderizado, facilitando futuras ampliaciones del proyecto.

### 5.8.2. Arquitectura final

Al finalizar el desarrollo, el proyecto quedó organizado en módulos especializados que cooperan entre sí manteniendo un bajo nivel de dependencia.

La Figura correspondiente a la arquitectura general muestra cómo la clase **Application** coordina el funcionamiento del renderizador, la interfaz gráfica, el sistema de partículas, el sistema meteorológico, el audio y los distintos elementos presentes dentro de la escena.

Esta organización facilita el mantenimiento del código y permite incorporar nuevos efectos visuales o componentes del entorno sin afectar significativamente el resto de la aplicación.

# Capítulo 6

## Resultados

En este capítulo se presentan los resultados obtenidos durante la ejecución de la aplicación desarrollada. Se analiza el comportamiento de cada uno de los componentes implementados y se verifica el cumplimiento de los objetivos propuestos al inicio del proyecto.

### 6.1. Escena final

La Figura 6.1 presenta el resultado final de la simulación.

Puede observarse la integración de todos los componentes desarrollados durante el proyecto:

- Sistema de partículas para representar la lluvia.
- Terreno texturizado.
- Árboles implementados mediante billboards.
- Cámara libre con desplazamiento en el plano XZ.
- Sonido ambiental de lluvia.
- Interfaz gráfica para modificar parámetros en tiempo real.
- Efectos meteorológicos de relámpagos.



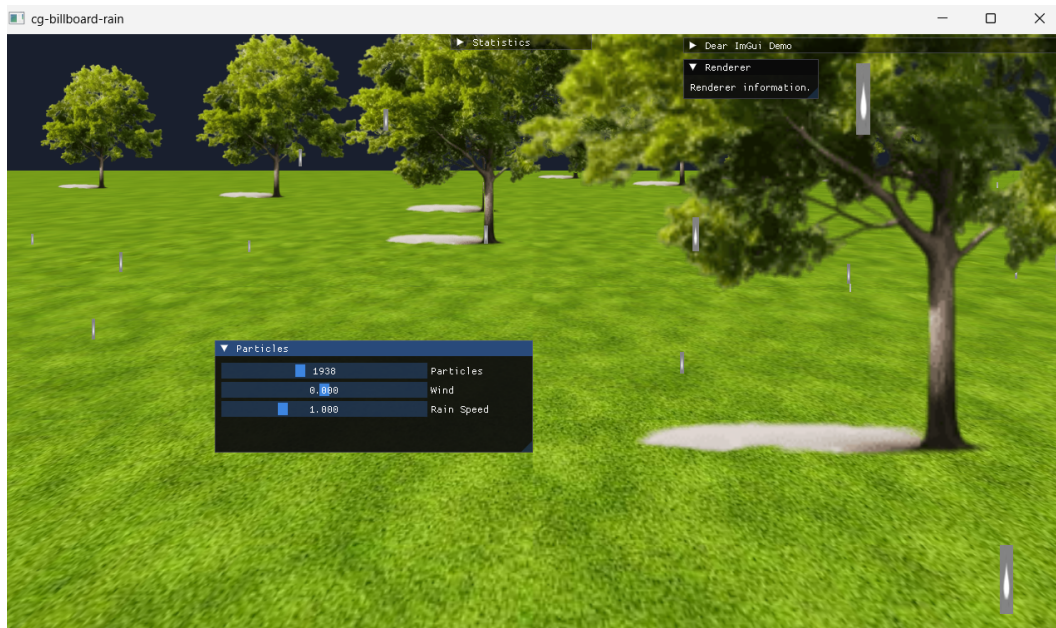


Figura 6.1: Resultado general de la simulación.

## 6.2. Sistema de partículas

El sistema de partículas logró representar una lluvia continua utilizando miles de partículas renderizadas simultáneamente.

Cada partícula posee un comportamiento independiente, incluyendo posición, velocidad y reinicio automático al abandonar la zona visible.

Gracias al uso de billboards, todas las partículas permanecen orientadas hacia la cámara, generando una representación visual consistente desde cualquier punto de observación.

La reconstrucción dinámica del sistema permite modificar la cantidad de partículas desde la interfaz gráfica sin necesidad de reiniciar la aplicación.

## 6.3. Sistema de árboles

Los árboles fueron implementados utilizando billboards de gran tamaño con texturas PNG que incluyen canal alfa.

Cada árbol mantiene siempre su orientación hacia la cámara, simulando modelos tridimensionales con un costo computacional muy reducido.

El algoritmo implementado evita el solapamiento entre árboles mediante una distancia mínima de separación.

Además, cuando un árbol abandona el radio de simulación alrededor del usuario, éste es reposicionado automáticamente delante de la cámara, manteniendo constante la densidad del bosque.

## 6.4. Terreno

El terreno consiste en un plano texturizado que sigue continuamente la posición horizontal de la cámara.

Este enfoque evita la necesidad de generar un escenario extremadamente grande, reduciendo considerablemente la cantidad de geometría procesada.

Desde la perspectiva del usuario, el terreno aparenta extenderse indefinidamente en todas las direcciones, produciendo una sensación de desplazamiento continuo.

## 6.5. Interfaz gráfica

La interfaz desarrollada con Dear ImGui permitió modificar diversos parámetros de la simulación durante su ejecución.

Entre los controles disponibles se encuentran:

- Cantidad de partículas.
- Intensidad del viento.
- Velocidad de caída de la lluvia.
- Información del rendimiento.
- Posición de la cámara.

Las modificaciones realizadas desde la interfaz se reflejan inmediatamente sobre la simulación, permitiendo experimentar distintos escenarios climáticos en tiempo real.

## 6.6. Sistema de audio

La incorporación de sonido ambiental incrementó significativamente la sensación de inmersión.

El audio de lluvia se reproduce continuamente durante la ejecución del programa.

Su intensidad y velocidad de reproducción cambian automáticamente de acuerdo con la cantidad de partículas presentes en la escena.

De esta manera, una lluvia más intensa produce también una percepción auditiva más fuerte y dinámica.

## 6.7. Sistema meteorológico

El sistema meteorológico implementado añade relámpagos aleatorios durante la simulación.

Cada relámpago incrementa temporalmente la iluminación del fondo mediante una modificación dinámica del color utilizado por el renderizador.

Posteriormente la intensidad disminuye progresivamente hasta recuperar el estado normal de la escena.

Este efecto visual contribuye a representar una tormenta de manera más realista sin incrementar significativamente el costo computacional.

## 6.8. Evaluación del rendimiento

Durante las pruebas realizadas se observó un comportamiento estable incluso utilizando varios miles de partículas simultáneamente.

La utilización de billboards para representar tanto la lluvia como los árboles permitió mantener un bajo consumo de memoria y una carga reducida sobre la GPU.

La interfaz gráfica mostró continuamente la cantidad de cuadros por segundo (FPS), permitiendo monitorear el rendimiento de la aplicación mientras se modificaban los parámetros de la simulación.

En equipos con aceleración gráfica compatible con OpenGL 3.3, la simulación mantiene una ejecución fluida para configuraciones de hasta 5000 partículas de lluvia.

## 6.9. Discusión

Los resultados obtenidos demuestran que es posible construir una simulación visualmente atractiva utilizando técnicas relativamente simples como billboards, sistemas de partículas y renderizado mediante OpenGL.

La arquitectura modular implementada facilitó el desarrollo incremental del proyecto y permitió incorporar nuevas funcionalidades sin modificar significativamente los

componentes existentes.

Asimismo, la integración del sistema de audio y los efectos meteorológicos incrementó el nivel de realismo alcanzado por la simulación, ofreciendo una experiencia más inmersiva para el usuario.

# Capítulo 7

## Conclusiones

Al finalizar el desarrollo del proyecto se alcanzaron satisfactoriamente los objetivos planteados para la simulación de una tormenta utilizando técnicas de gráficos por computadora basadas en OpenGL.

1. Se implementó un sistema de partículas capaz de representar lluvia en tiempo real mediante billboards, permitiendo controlar dinámicamente la cantidad de partículas, su velocidad y el efecto del viento.
2. Se desarrolló una arquitectura modular que separa claramente las responsabilidades de renderizado, escena, partículas, audio, interfaz gráfica y efectos meteorológicos, facilitando el mantenimiento y la futura expansión del proyecto.
3. Se implementó un escenario compuesto por un terreno texturizado y árboles representados mediante billboards, optimizando el rendimiento al evitar modelos tridimensionales complejos.
4. Se integró un sistema de audio ambiental utilizando la biblioteca Miniaudio, sincronizando automáticamente el volumen y la velocidad del sonido con la intensidad de la lluvia representada por el sistema de partículas.
5. Se incorporó un sistema meteorológico sencillo basado en relámpagos aleatorios que modifican temporalmente la iluminación general de la escena, incrementando la sensación de realismo sin afectar significativamente el rendimiento.
6. La utilización de Dear ImGui permitió construir una interfaz gráfica intuitiva para modificar en tiempo real los parámetros principales de la simulación y monitorear información de rendimiento como los cuadros por segundo (FPS).

7. Durante las pruebas realizadas, la aplicación mantuvo un comportamiento estable incluso utilizando miles de partículas simultáneamente, demostrando que la utilización de billboards constituye una alternativa eficiente para representar fenómenos naturales como lluvia y vegetación.

En conjunto, el proyecto permitió aplicar diversos conceptos fundamentales de gráficos por computadora, entre ellos transformaciones geométricas, matrices de vista y proyección, texturizado, transparencia mediante canal alfa, renderizado de billboards, sistemas de partículas, integración de audio y organización modular de aplicaciones desarrolladas con OpenGL.

# Capítulo 8

## Trabajo futuro

Aunque el proyecto cumple satisfactoriamente con los objetivos propuestos, existen diversas mejoras que podrían incorporarse para incrementar el nivel de realismo y ampliar las funcionalidades de la simulación.

- Implementar sombras dinámicas generadas por los relámpagos utilizando mapas de sombras (Shadow Mapping).
- Incorporar efectos de niebla y volumetría para representar condiciones climáticas más realistas durante tormentas intensas.
- Agregar animaciones de balanceo en los árboles en función de la intensidad y dirección del viento.
- Implementar múltiples tipos de precipitación, como nieve, granizo o lluvia ligera, reutilizando la arquitectura del sistema de partículas.
- Incorporar sonidos ambientales adicionales, incluyendo viento, truenos con diferentes intensidades y sonidos producidos por la lluvia al impactar distintas superficies.
- Desarrollar un sistema de clima dinámico capaz de alternar automáticamente entre cielo despejado, lluvia, tormenta y otros estados meteorológicos.
- Optimizar el renderizado utilizando técnicas modernas de OpenGL como Instanced Rendering para representar un número considerablemente mayor de partículas y vegetación.

- Implementar un sistema de terreno basado en alturas (Height Maps) que permita representar montañas, colinas y desniveles en lugar de un plano completamente uniforme.
- Incorporar modelos tridimensionales para algunos elementos del escenario, combinándolos con billboards para mantener un adecuado equilibrio entre calidad visual y rendimiento.
- Desarrollar un sistema de iluminación física (PBR) que mejore el comportamiento de los materiales frente a la luz generada por los relámpagos.
- Añadir soporte para cámaras completamente libres controladas mediante el mouse, permitiendo explorar la escena desde cualquier ángulo.
- Incorporar un ciclo completo de día y noche con variaciones automáticas de iluminación y condiciones climáticas.

La arquitectura modular implementada durante este proyecto facilita la incorporación de estas mejoras, ya que cada componente se encuentra desacoplado del resto del sistema, permitiendo extender la aplicación sin necesidad de modificar significativamente la estructura existente.



# Capítulo 9

## Repositorio

Link del repositorio: <https://github.com/DiegoRivas1/cg-billboard-rain.git>

# Bibliografía

- [1] J. de Vries, *Learn OpenGL*, 2024. [Online]. Available: <https://learnopengl.com>
- [2] D. Shreiner, *OpenGL Programming Guide*. Addison-Wesley, 2013.
- [3] G. Sellers and R. Wright, *OpenGL SuperBible*. Addison-Wesley, 2015.
- [4] T. Akenine-Möller, E. Haines, and N. Hoffman, *Real-Time Rendering*, 4th ed. CRC Press, 2018.
- [5] “Glfw documentation,” 2025. [Online]. Available: <https://www.glfw.org/documentation.html>
- [6] “Glad opengl loader,” 2025. [Online]. Available: <https://glad.dav1d.de>
- [7] O. Cornut, “Dear imgui,” 2025. [Online]. Available: <https://github.com/ocornut/imgui>
- [8] S. Barrett, “stb image,” 2025. [Online]. Available: <https://github.com/nothings/stb>
- [9] D. Reid, “Miniaudio,” 2025. [Online]. Available: <https://miniaud.io>